

70. Des ennemis qui vivent et qui meurent

Les méthodes, dans un vrai tour de boucle.

1. Ce qu'il faut comprendre

`hasard()` lit le compteur libre du matériel : un nombre imprévisible, sans table ni graine. `semer(n)` permet de choisir son point de départ quand on veut au contraire une suite reproductible.

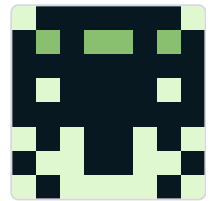
`cache(n)` ôte le lutin de l'écran. Sans lui, un ennemi mort resterait affiché, immobile — ce qui se remarque tout de suite, heureusement.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1782 octets** de cartouche.

1. UN DESSIN — ENNEMI

```
Tuile ENNEMI = {  
    "#####.", "#-#-#-#", "#####", "#.####.#",  
    "#####", ".#.#.#.", "#..#.#.", "#...#.",  
};
```



à l'écran

2. UN ENREGISTREMENT — MONSTRE

```
struct Monstre {  
    uint8_t x, y, vie;  
  
    uint8_t vivant() { return vie > 0; }  
    void avancer()   { if (vivant()) x++; }  
    void tuer()      { vie = 0; }  
};
```

3. UN TABLEAU — TROUPE

```
Monstre troupe[3];
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    for (uint8_t i = 0; i < 3; i++) {
        troupe[i].x = 10 + i * 30;
        troupe[i].y = 40 + i * 24;
        troupe[i].vie = 1;
    }

    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);
        if (appui && !avant) troupe[hasard() % 3].tuer();
        avant = appui;

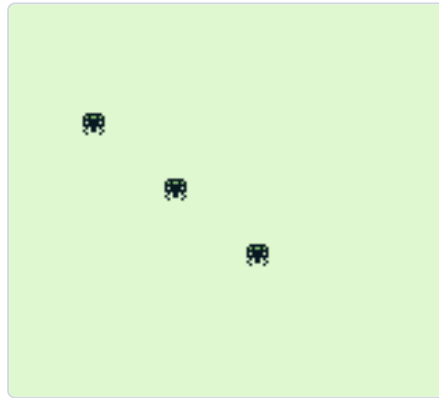
        for (uint8_t i = 0; i < 3; i++) {
            troupe[i].avancer();
            if (troupe[i].vivant()) sprite(i, troupe[i].x, troupe[i].y, ENNEMI);
            else cacher(i);
        }
        image();
    }
}
```

3. Ce qui se passe, étape par étape

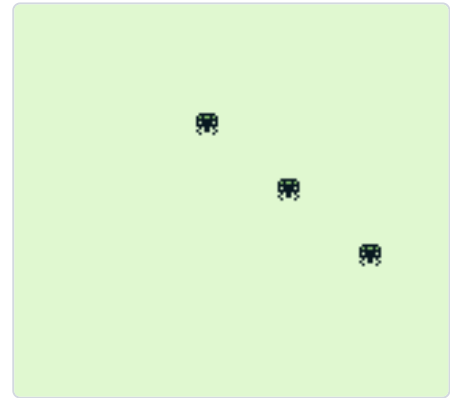
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



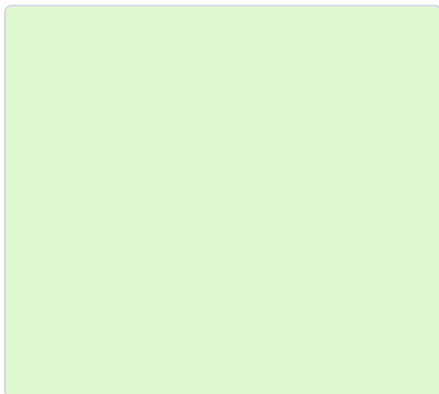
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Trois monstres qui avancent ; A en fait disparaître un au hasard.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les trois sont posés
- ✓ ils avancent
- ✓ A finit par tous les tuer (0 encore à l'écran)
- ✓ et cacher() les a bien ôtés de l'écran

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

ENNEMI

tuile n° 44 — poser(x, y, ENNEMI)

- zoom22×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile



taille réelle — 8 × 8 pixels

Ensuite : **71. Les collisions** — niveau 9 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++